

Audion Get Tools

Портативный CMD/PowerShell-инструментарий для установки, обновления, проверки, экспорта и импорта WinGet-пакетов на рабочих станциях Audion.

Главная точка входа: `Launcher-Audion-Get.cmd`. GUI-точка входа: `launcher_gui.cmd`. Для русскоязычного интерфейса используйте `Launcher-Audion-Get-RU.cmd`.

Что изменено в текущей схеме

- Деление на `common` и `extended` упразднено.
- Все массовые установки и обновления идут через тематические списки.
- Любой запуск списка проходит через контролируемый режим `Y/N/Q`, где `Enter` означает `Yes`.
- В GUI этот сценарий заменён группами чекбоксов: пользователь заранее отмечает нужные WinGet ID, затем запускаются только выбранные пакеты без консольных `Y/N/Q`.
- GUI показывает установленные пакеты как `Имя программы | ID`, чтобы удаление оставалось узнаваемым и точным.
- Для удаления в GUI есть групповые чекбоксы по тем же тематическим блокам, что установка/обновление, плюс `Custom / пользовательские` и `Прочее установленное`.
- Некоторые деинсталляторы открывают UAC или своё окно подтверждения; GUI ждёт выбор пользователя, проверяет результат через `winget list --id <ID> -e` и продолжает пакетную операцию.
- Между основной частью GUI и терминалом есть перетаскиваемый разделитель; размер терминальной панели запоминается.
- После процессов окно не закрывается сразу: используется `pause`, затем лаунчер возвращается в своё меню.
- Отмена FZF через `Esc` возвращает в меню без ошибки.
- Скриптовые папки перенесены из корня в `system_core\winget\`.
- MSVC-лаунчер оставлен только для проверенного сценария: установка MSVC Legacy, установка MSVC 2015+, обновление только MSVC 2015+ x86/x64.

Структура

```
Audion Get Tools\  
  Launcher-Audion-Get.cmd  
  launcher_gui.cmd  
  Launcher-Audion-Get-RU.cmd  
  cli\Launcher-Audion-MSVC-Legacy.cmd  
  cli\Launcher-Audion-MSVC-Legacy-RU.cmd  
  cli\Launcher-Audion-Tools.cmd  
  cli\Launcher-Audion-Tools-RU.cmd  
  cli\Launcher-Audion-Check-Apps.cmd  
  cli\Launcher-Audion-Check-Apps-RU.cmd  
  cli\Launcher-Audion-Install-Apps.cmd  
  cli\Launcher-Audion-Install-Apps-RU.cmd  
  cli\Launcher-Audion-Update-Apps.cmd  
  cli\Launcher-Audion-Update-Apps-RU.cmd  
  config\  
    system.txt  
    dev.txt  
    ai.txt  
    pkms.txt  
    office.txt  
    media.txt  
    browsers-vpn.txt  
    hardware-benchmarks.txt  
    custom.txt  
    pins.txt  
  system_core\  
    fzf.exe  
    powershell\  
    winget\  
      scripts\  
      package_apps\  
      install_apps\  
      check_apps\  
      export_import\  
      msvc_legacy_updates\  
  install\  
  licenses\  
  logs\  
  release\  
  .runtime\
```

Тематические списки

- `config\system.txt` — рантаймы, терминалы, архиваторы.
- `config\dev.txt` — инструменты разработки.
- `config\ai.txt` — AI-инструменты.
- `config\pkms.txt` — PKMS, заметки, базы знаний.
- `config\office.txt` — офис, документы и чтение.
- `config\media.txt` — изображения, звук и видео; секции разделены пустыми строками.
- `config\browsers-vpn.txt` — браузеры, VPN, сетевые и коммуникационные клиенты.
- `config\hardware-benchmarks.txt` — диагностика железа, бенчмарки, служебные утилиты.
- `config\custom.txt` — пользовательские ID, установленные через GUI «Установить по ID» или добавленные вручную.
- `config\installed_uninstall_aliases.yaml` — установленные ARP/MSIX/runtime ID и маски для группировки чекбоксов удаления и группового обновления.
- `config\pins.txt` — пакеты для `winget pin`.

В PKMS порядок намеренный: сначала `Notion.Notion` и `Notion.NotionCalendar`, затем Obsidian, Joplin, Evernote и остальные PKMS-инструменты. Office/документы вроде LibreOffice, Acrobat Reader и Calibre живут в отдельной группе Office.

GUI

`launcher_gui.cmd` запускает NiceGUI/pywebview-оболочку с чекбоксами, живым терминалом и строкой ручных команд.

- Установка сканирует установленные ID и показывает только отсутствующие пакеты проекта; уже установленное повторно не ставится.
- Для обновлений есть четыре пути: предпросмотр текущего `winget upgrade` как `Name | ID | current -> available`, обновить всё, что вернул `winget upgrade`, обновить видимые доступные обновления по группам проекта или обновить отмеченные пункты из плоского списка доступных обновлений. Блоки без доступных обновлений показывают понятную плашку `Обновления отсутствуют`.
- В блоках чекбоксов есть фильтр по имени/ID и кнопки отметить/снять. При активном фильтре кнопки работают только по видимым совпадениям.
- Раздел `Установка, удаление по ID` поддерживает поиск, добавление найденного точного ID в выбранный список/GUI-группу, установку/удаление введённого ID и групповое удаление

установленных ID.

- В группе **Система** есть **.NET Framework 3.5** — это компонент Windows, а не WinGet-пакет: Audion Get Tools включает его через DISM (полезная нагрузка приходит из Windows Update) и убирает из списка, когда компонент уже включён.
- Окно **Portable** начинается с кнопки **Google Chrome (веб-установщик)**: скачивается веб-установщик Google (~12 МБ) под язык и разрядность текущей системы и ставится без вопросов.
- У каждого чекбокса пакета есть маленькие кнопки внутри самой карточки: зелёная стрелка вниз скачивает установщик WinGet в **output\Downloads**, ничего не устанавливая, а синяя стрелка открывает страницу, где сборку можно выбрать руками — GitHub **releases/latest**, TechPowerUp или страницу загрузки разработчика.
- Малиновая кнопка-архив скачивает zip или standalone-сборку — **winget download --installer-type zip|portable** — в **output\Portable\<Имя программы>**. Она рисуется только для ID из таблицы **PACKAGE_ARCHIVE_TYPES**: наличие архива у каждого пакета проверено через **winget show --installer-type**, а типы вроде **inno (zip)** в таблицу не попали — это установщик внутри архива.
- Вторая таблица, **PACKAGE_GITHUB_BUILDS**, — для случая, когда сборка у разработчика есть, а в манифесте WinGet её нет: **winget download** такой файл не отдаст никогда. Значение — репозиторий и два регулярных выражения, для установщика и для архива (**Eugeniy/tabby** → **tabby-[\d.]+\--setup-x64\.exe** и **tabby-[\d.]+\--portable-x64\.zip**). Обе кнопки берут файл из одного выпуска, поэтому не расходятся в версиях: релиз попадает на GitHub раньше, чем обновляется манифест. Вторая запись — **Zarestia-Dev/rcclone-manager**: в манифесте WinGet лежит только msi, и он отстаёт от выпуска, а портативной сборки там нет вовсе. Разрядность в именах у этого разработчика написана по-разному — **x64** у установщика и **x86_64** у портативного архива, — поэтому у каждой кнопки своё выражение, а не одно общее.
- Запасной путь для всех остальных: если **winget download** ничего не отдал, а страница пакета ведёт на GitHub, файл ищется в последнем выпуске этого репозитория. Нужную сборку выбирает **pick_windows_asset**: отсекает другие системы и разрядности, контрольные суммы, символы отладки и **.blockmap**, предпочитает явный x64, а для архива — сборку со словом **portable**. Не нашлось однозначного файла — возвращается пусто, и человек видит обе причины отказа, а не догадку.
- Лимит GitHub API — 60 анонимных запросов в час, и он выбирается незаметно. Поэтому список файлов выпуска читается сначала через API, а при любой его ошибке — со страниц **releases/latest** и **releases/expanded_assets/<tag>**, где квоты нет вовсе. Ссылка на

файл берётся из того же ответа, так что между «нашли» и «скачали» GitHub больше ни о чём не спрашивают.

- Раскладка загрузок: установщики ложатся плоско в `output\Downloads` (включая `ChromeSetup.exe`), архивы и standalone — в `output\Portable\<Имя>`, наборы для установки — в `output\Install\<Имя>`. Имя папки берётся из подписи чекбокса и сохраняет написание разработчика (`MPC-BE`, `Notepad++`, `MSVC All-in-One (TechPowerUp)`); ID WinGet в путях не используется. Нажатие на них не переключает чекбокс, а кнопка страницы работает и во время запущенного пакетного прогона.
- Зачем и то, и другое: WinGet ставит ровно то, что лежит в манифесте, и молча. Кроссплатформенные и портативные сборки, опции установщика (например, пункты контекстного меню Explorer у PowerShell) и самообновление через Microsoft Update живут на странице разработчика. `Microsoft.PowerShell` — самый явный случай: в WinGet у него только MSIX-пакет, поэтому MSI с этими опциями берётся с GitHub.
- В `Классических скриптах` есть оба all-in-one набора VC++ — TechPowerUp и `abbodi1406/vcredist`, у каждого кнопка скачивания и синяя стрелка на страницу свежего выпуска. Оба содержат VC++ 2012, которого в WinGet нет вовсе. Audion Get Tools только скачивает и распаковывает; запуск `install_all.bat` или `VisualCppRedist_AI0.exe /ai` остаётся ручным шагом с правами администратора.
- MSVC разделён на две группы установки: `MSVC рантайм (2015+)` — единственное, что нужно современной системе, и `Advanced: MSVC 2005–2013` — для старого софта. В подсказках обеих рекомендуются all-in-one наборы: они ставят всё семейство одним заходом и включают 2012. В удалении все рантаймы VC++ по-прежнему в одном блоке.
- В `Служебных процедурах` есть `Проверить рантаймы MSVC`: реальные версии из реестра, сравнение семейства 2015+ с WinGet, полный список записей в «Программах и компонентах» и честная пометка, что 2012 закрывается только all-in-one наборами.
- В `Классических скриптах` оба набора можно и установить. Эти действия сначала спрашивают подтверждение, требуют прав администратора и пишут состояние реестра рантаймов до и после, потому что `install_all.bat` сообщает об успехе при любом исходе.
- В окне `Portable` рядом с `Google Chrome (веб-установщик)` на той же строке стоит иконка «только скачать».
- В окнах с параметрами кнопка запуска называется по действию — `УСТАНОВИТЬ`, `ОБНОВИТЬ`, `УДАЛИТЬ`, `НАЙТИ`, `ЗАКРЕПИТЬ` — с зарезервированной шириной.
- `AI Package Planner` — необязательный child-раздел для LLM-подбора пакетов. Он управляет локальными ключами/моделями/prompt, проверяет предложения через WinGet

search, пишет план для ревью и позволяет запускать только выбранные точные проверенные ID через обычный путь Audion.

- В **AI Package Planner**: **Задача для AI** — что нужно найти или подготовить, **Шаблон инструкции** — сохранённый пресет поведения AI, **Инструкция AI-планировщика** — правила, как AI должен думать, проверять WinGet ID и оформлять план.
- Групповое удаление раскладывает установленные пакеты по тематическим блокам; неизвестные пользовательские ID попадают в **Custom**, а всё вне списков проекта — в **Прочее установленное**.
- Protected uninstall помечает и требует отдельный флажок для App Installer, Terminal, PowerShell, MSVC/.NET/WindowsAppRuntime, VCLibs, WSL, Microsoft Edge и похожих общих зависимостей.
- ARP/MSIX/runtime-алиасы можно привязать к группам удаления и группового обновления, не добавляя эти ID в списки установки.
- Поиск установленных пакетов загружается асинхронно и использует общий кэш, чтобы GUI не зависал на больших списках WinGet. Окно группового удаления делает один сериализованный **winget list**, а затем раскладывает общий снимок по тематическим блокам.
- При установке неизвестного ID через GUI он добавляется в **config\custom.txt**.
- **Добавить ID в список** пишет ID в выбранный **config*.txt** и, для GUI-групп, добавляет такой же элемент в **config\tool_manifest.yaml**, чтобы он появился обычным чекбоксом.
- Удаление не останавливается на первом проблемном ID: GUI проходит выбранный список, а неудачные ID сообщает в конце.
- Для UAC или внешних GUI-деинсталляторов GUI пишет **[WAIT]**, ждёт до таймаута и опрашивает **winget list --id <ID> -e**.
- Обслуживание находится в корневом child-разделе **Служебные процедуры**. Там есть **Очистить логи**: удаляет старые файлы из **logs**, но сохраняет лог текущей операции; **report** не трогается.
- **Health / Doctor** проверяет доступность WinGet, источники, количество установленных/обновлений и GUI doctor прямо из приложения.
- **Health / Doctor** показывает и MCP-сервер WinGet (**winget mcp**, **WindowsPackageManagerMCPServer.exe**). Пакеты через него не запускаются: его два инструмента, **find-winget-packages** и **install-winget-package**, — подмножество путей по точному ID, а через MCP терялись бы живой прогресс, логи, отчёты и проверка защищённых ID.

- Проверить наличие ID оставлено там как диагностическое действие; обычные сценарии установки/обновления уже сканируют отсутствующие и обновляемые пакеты.
- Пакетные действия пишут `action_summary` в JSON/Markdown: выбранные пакеты, статусы, ошибки, пропущенные обновления и ожидание внешних деинсталляторов.
- Терминальная панель сохраняет ANSI-цвет, UTF-8/кириллицу и имеет перетаскиваемый разделитель с основной частью окна.
- WinGet запускается через псевдоконсоль Windows (ConPTY), поэтому в журнал попадают живые полосы загрузки и цвета, как в обычном PowerShell. Отключить можно переменной `AUDION_GET_CONSOLE_STREAM=0`.
- Прогресс скачивания — одна живая строка со спиннером, которая обновляется на месте: загрузка на 300 МБ больше не забивает лог сотнями строк. В файловый лог пишется редкий след — раз в 1, 5 или 10 секунд, в зависимости от того, больше ли загрузка 10, 50 или 100 МиБ.
- Сканирование обновлений видит все установленные пакеты, включая те, у которых WinGet не может прочитать установленную версию (большинство браузеров), а таблица обновлений разбирается по позициям колонок, поэтому русская Windows показывает тот же список, что и английская.
- Обновление `Microsoft.AppInstaller` подменяет сам `winget`: после такого пакета GUI ждёт возвращения алиаса и при необходимости переключается на реальный путь пакета в `Program Files\WindowsApps`, вместо падения с `WinError 1920`.
- Поэтому App Installer больше не чекбокс: кнопка «Обновить WinGet» стоит вверху окон обновления, запускается отдельно и предупреждает, что после неё Audion Get Tools нужно перезапустить.
- Действия в окнах команд собраны в группы с заголовком и фоном; ни одно поле, фильтр или селект не остаётся на голом фоне.
- Окно команд не открывается ради ссылок на другие окна: разделы вроде `Установка,` `удаление по ID` и `Импорт / экспорт` сразу показывают свои команды на месте — каждая как группа со своими полями и кнопкой с именем самой команды (`Поиск`, `Добавить ID в список`, `Установить по ID`, `Удалить по ID`), а не общим `Запустить`. Отдельными окнами остаются только тяжёлые списки чекбоксов. Поле, повторяющееся в нескольких группах (например WinGet ID), везде хранит одно значение.
- Состояние сплиттера хранится в профиле ruwebview/браузера. Для сброса к релизным значениям удалите `._runtime\webview` или выполните релизную очистку, которая удаляет `._runtime`; не удаляйте `runtime\`, там лежит портативный Python. Терминал по умолчанию — `500px`; встроенный размер окна — `1600x900`.

Лаунчеры

`Launcher-Audion-Get.cmd` — основной пользовательский вход. Он запускает установку, обновление и пины по тематическим спискам, а также открывает MSVC и Tools.

`cli\Launcher-Audion-Tools.cmd` — служебный лаунчер. Он оставлен для export/import, проверки отдельных приложений, точечной установки/обновления из всех тематических списков, удаления пакета из списков и поиска в реестре WinGet.

`cli\Launcher-Audion-MSVC-Legacy.cmd` — MSVC-лаунчер. Legacy-обновления 2005-2013 намеренно не запускаются; обновляется только MSVC 2015+ x86/x64 через `system_core\winget\install_apps\Update-Audion-MSVC-2015+.cmd`.

Все перечисленные лаунчеры имеют RU-версии с суффиксом `-RU.cmd`. `Exit` / `Q` закрывает текущий лаунчер, а завершение дочернего процесса возвращает в меню этого лаунчера.

Правила запуска списков

Массовые операции проходят через:

```
system_core\winget\scripts\Launch-WinGet-Lists.cmd
system_core\winget\scripts\Run-WinGet-From-Lists.ps1
```

Режим подтверждения:

```
Enter/Y = установить или обновить
N        = пропустить пакет
Q        = завершить текущий список
```

Логи пишутся в `logs\`, временные файлы меню — в `._runtime\`. RU-лаунчеры используют отдельные временные файлы с суффиксом `_ru`, чтобы не конфликтовать с EN-версиями.

Канонические названия Workbench

Верхняя панель I/O во всех Audion-проектах использует один словарь: `Источник`, `Добавить файл...`, `Назначение`, `Сбросить`, `Удалить`, `Список`.

- `Источник` и `Назначение` открывают текущие маршруты.
- `Добавить файл...` выбирает один файл как текущий Источник без копирования.

- **Сбросить** удаляет незакреплённую историю и возвращает проектные **input/output** ; файлы не удаляются, закреплённые пути сохраняются.
- **Удалить** после общего подтверждения очищает текущие Источник и Назначение.
- **Список** выводит текущий файл либо список файлов текущей папки Источника.

Адресная корзина очищает содержимое выбранного маршрута. Для внешнего Источника требуется отдельное подтверждение; корень диска и корень проекта защищены.